

Rebort

How a Heap Works

A heap is a special tree-based data structure, usually represented as an array(vector in our case). It follows the heap property:

In a **max-heap**, each parent node is greater than or equal to its children.

In a **min-heap**, each parent node is less than or equal to its children.

A heap is always a complete binary tree, which means all levels are filled from left to right except possibly the last one.

The smallest or largest element is always at the root, depending on the heap type.

Why It Is Efficient

A heap is efficient because it allows fast access to the minimum or maximum element.

Getting the top element is $O(1)$

Inserting an element is $O(\log N)$

Removing the top element is $O(\log N)$

This makes heaps very useful when we need to repeatedly find the smallest or largest item, such as in priority queues.

Comparison with other structures

Compared with other data structures, heaps have different advantages:

- **Versus arrays:** Arrays are simple and good for random access, but finding the minimum or maximum may take $O(N)$. A heap gives faster access to the top priority element.
- **Versus linked lists:** Linked lists are easy to insert into, but searching is slow. Heaps are better when priority matters.
- **Versus binary search trees:** Trees can also keep data ordered, but heaps are simpler and faster for priority operations.

Comparison

Time Complexity

Operation	Manual Heap	STL priority_queue	Why
Insertion	$O(\log N)$	$O(\log N)$	Both insert in the end of the queue then apply HeapifyUp
Pop	$O(\log N)$	$O(\log N)$	Both insert last element in the top of the queue then apply HeapifyDown
View The Highest	$O(1)$	$O(1)$	Both read the element in the index 0 directly
Update Element in The Queue	$O(\log N)$	Cannot do it	$O(1)$ To find the element and $O(\log N)$ to HeapifyUp or HeapifyDown

Ease of Use

Manual Heap:

- **Advantage:** Full access to any element
- **Disadvantage:** Need to Write the full structure line by line

STL priority_queue:

- **Advantage:** No need to write the any structure only one line to be ready
- **Disadvantage:** No access to any element after insert it only the highest element by pop it

Real Time Complexity Comparison

Test Size (Patients)	Manual Heap	STL priority_queue
100K	97.1 <i>ms</i>	136.7 <i>ms</i>
1M	1211.7 <i>ms</i>	1613.5 <i>ms</i>
10M	15816.6 <i>ms</i>	20154.2 <i>ms</i>
50M	91181.1 <i>ms</i>	112637 <i>ms</i>

Tests shows that the Manual Heap have less time than STL priority_queue by 20% to 30%

All tests are done on the same machine at the same time

The results above are the average of 10 operations for each Test size